

Operating System — INSY 8223

Group 3 Assignment Answers

Members:

1. MUTANGANA Joseph 29061
2. Tesi Divine 26017
3. INEZA keissy AKEANE ID 28770
4. Manzi keith 28799
5. ISHEMA NGABO Ange 26035
6. Azabe benine 28291
7. MIZERO NSHUTI Olivier 28333

1. Circular Wait Condition

Circular wait is one of the four necessary conditions for deadlock. It occurs when a set of processes are waiting for each other in a circular chain. For example, Process P1 is waiting for a resource held by P2, P2 is waiting for a resource held by P3, and P3 is waiting for a resource held by P1. This creates a cycle, and none of the processes can proceed.

Resource Allocation Graph (description):

$P1 \rightarrow R1 \rightarrow P2 \rightarrow R2 \rightarrow P3 \rightarrow R3 \rightarrow P1$ (cycle formed)

2. Banker's Algorithm

The Banker's Algorithm is a deadlock avoidance algorithm that determines whether a system is in a safe state before allocating resources. It simulates resource allocation and checks if all processes can complete without leading to deadlock.

Purpose:

- Ensures the system never enters an unsafe state
- Allocates resources only if safe sequence exists

3. User-level vs Kernel-level Threads

User-level threads:

- Managed by user libraries
- Faster to create and switch
- Kernel is unaware of them

Advantages:

- Fast context switching
- Low overhead

Kernel-level threads:

- Managed by the OS kernel
- Each thread is known to OS

Advantages:

- True parallelism
- Better scheduling by OS

4. pthread_join()

pthread_join() is used to make the calling thread wait until a specified thread finishes execution.

Importance:

- Ensures synchronization
- Prevents premature termination of main thread
- Allows retrieval of thread results

5. Benefits of Multithreading

1. Responsiveness: A UI application remains responsive while performing background tasks.

Example: A browser loading a page while allowing scrolling.

2. Resource Sharing: Threads share memory.

Example: Multiple threads accessing shared data in a program.

3. Economy: Lower overhead than processes.

Example: Creating threads is faster than processes.

6. Static vs Dynamic Loading

Static Loading:

- All program code loaded into memory before execution

Dynamic Loading:

- Code loaded only when needed during execution

Dynamic loading is preferred when:

- Memory is limited
- Large programs with unused modules

7. External Fragmentation

External fragmentation occurs when free memory is scattered in small blocks, making allocation difficult.

Compaction:

- Rearranges memory to combine free spaces into one block

Cost:

- High CPU overhead
- Requires relocating processes

8. Single vs Multiple Partition Allocation

Single-partition:

- Entire memory allocated to one process

Multiple-partition:

- Memory divided into partitions for multiple processes

More flexible:

- Multiple-partition allocation

9. Page Fault

A page fault occurs when a process tries to access a page not currently in memory.

Steps:

1. OS detects page fault
2. Locate page on disk
3. Find free frame
4. Load page into memory
5. Update page table
6. Resume execution

10. Segmentation vs Paging Visibility

Segmentation is visible to the user because it reflects logical divisions (code, data, stack).

Paging is not visible because it is handled by the OS and divides memory into fixed-size pages without user awareness.